



Tutorial S1D: Stack (Solutions)

1. A stack is a Last In, First Out (LIFO) data structure. It has a pointer called **top** and the operations **push** and **pop**.

- a) What is meant by a LIFO data structure?

LIFO data structure means that the data that is inserted last will be the first to be removed. The data that is inserted last will stay in the data structure for the shortest amount of time.

- b) Name and describe two types of errors that could occur when attempting to add a data item or remove a data item from the stack.

Overflow error – this error happens when the stack is full and further attempts to add data item will lead to an overflow. Hence, item cannot be added and error will occur.

Underflow error – this error happens when the stack is empty and there is no data item to remove from the stack. Hence, error will occur.

- c) Describe the operations push and pop.

Push operation is to add a data item on the stack. Before pushing, the stack needs to be checked for overflow.

Pop operation is to remove an item from the stack. Before popping, the stack needs to be checked for underflow.

- d) Write pseudocodes to implement the push and pop operations using a fixed size array.

Refer to pages 8 and 9

<pre>PROCEDURE push(item) IF isFull THEN print ("Stack is full") ELSE top ← top + 1 stack[top] ← item ENDIF ENDPROCEDURE</pre>	<pre>FUNCTION pop IF isEmpty THEN print ("Stack is empty") ELSE item ← stack[top] top ← top - 1 RETURN item ENDIF ENDFUNCTION</pre>
--	---

- e) Besides push and pop, other operations required to implement stack as an array are peek, isEmpty and isFull. Fill in the table when the operations are carried out on the stack s.

Stack operation	Stack contents	Return value
s.isEmpty()	[]	True
s.push('Cycle')	['Cycle']	
s.push('Jump')	['Cycle', 'Jump']	
s.push('Walk')	['Cycle', 'Jump', 'Walk']	
s.isEmpty()	['Cycle', 'Jump', 'Walk']	False
s.peek()	['Cycle', 'Jump', 'Walk']	'Walk'
s.pop()	['Cycle', 'Jump']	'Walk'
s.pop()	['Cycle']	'Jump'



2. A stack is to be implemented using an array of 20 elements.

a) Describe an algorithm to remove an item from a stack and place it in a variable x.

Function Pop will remove item from stack and return the item. First, check for underflow. If underflow, print error message "Stack is empty" and exit. Read item from top of stack and assign to variable X. Decrement top pointer of stack by 1. Return variable X.

```
FUNCTION Pop(stack)
IF stack.top = None THEN
    PRINT "Stack is empty"
ELSE
    X ← stack.top
    stack.top ← stack.top - 1
ENDIF
RETURN X
ENDFUNCTION
```

b) With the aid of examples, explain what nested functions or nested subroutines are.

A nested function (also called inner function) is a function defined inside another function. The outer function can call the inner function. Nested function are often used to organise code or restrict the scope of the inner function.

```
def functionA(): # outer function
    print ("Hello from outer function")
    def functionB(): # nested/inner function
        print ("Hello from inner function")
    functionB()

functionA()

def greet(): # outer function
    def say_hello(): # nested/inner function
        print ("Hello!")
    say_hello()

greet()
say_hello is a nested function inside greet.
```

c) Explain with the aid of diagrams or otherwise, how a stack can be used by the operating system to process "nested functions" or "nested subroutines".

One common application of stacks in an operating system is to manage nested function or subroutine calls using a runtime stack (also called a call stack).

When a function (or subroutine) is called, a stack frame is created and pushed onto the stack. This stack frame stores:



- The return address (where program should continue after the subroutine finishes)
- The subroutine's parameters
- Its local variables

If the subroutine then calls another subroutine (i.e. a nested call), another stack frame is pushed onto the stack. This continues for every new function call. When a subroutine finishes execution, its stack frame is popped from the stack, and control returns to the return address stored in the previous frame.

This use of the stack ensures that each function's context is preserved and that the program can return correctly, even with deeply nested function calls.

(Refer to page 11 for diagrams.)

- d) Outline the data attributes and member functions for a class stack abstract data type. You need not go into details as to how they will be implemented.

```
// Assume an integer STACK
Class Stack
Private attributes:
    item : integer
    top : integer
    max_size : integer
Public methods:
    Stack(stack_size: integer) //Constructor
    IsEmpty(): boolean
    IsFull(): boolean
    Pop(): integer
    Push()
    getStacksize(): integer
```

3. A stack is implemented using a linked list. Each node contains a data field and a next pointer. The top of the stack is pointed to by the variable head.

- a) Write pseudocode for the push and pop operations.

```
PROCEDURE push(item)
    newNode ← NEW Node
    newNode.data ← item
    newNode.next ← head
    head ← newNode
ENDPROCEDURE

PROCEDURE pop
    IF head == NULL THEN
        OUTPUT "Stack Underflow"
        RETURN NULL
    ENDIF
    item ← head.data
    head ← head.next
    RETURN item
ENDPROCEDURE
```



- b) Compare the use of a linked list versus an array to implement a stack. State one advantage and one disadvantage of the linked list implementation.

Advantage	The stack implemented using a linked list can grow or shrink dynamically without the need for resizing, unlike an array with a fixed size.
Disadvantage	A linked list requires additional memory to store pointers, and accessing or allocating memory dynamically may result in slower performance compared to arrays.

4. Given the following pseudocode to implement the pop operation for a stack based on a linked list:

```
PROCEDURE pop()  
  IF head == NULL THEN  
    RETURN NULL  
  END IF  
  temp ← head  
  head ← head.next  
  RETURN temp  
END PROCEDURE
```

- a) Identify two potential issues in this implementation.

Returns the entire node instead of the value:

- RETURN temp returns the whole node, not just the data. This may not be the expected behavior.

Silent failure on empty stack:

- RETURN NULL is used when the stack is empty, but there's no clear message or handling of the underflow condition.

- b) Suggest improvements to ensure robustness.

Return only the data (not the node).

Provide clear feedback if the stack is empty.